

# IMAGE EXTRACTION FROM PDF DOCUMENTS

Extracting Charts, Graphs and Images from PDFs

## PDF Image / Chart / Graph Extraction Workflow

Embedded images are extracted directly; vector/scanned content can be rendered and processed.



**B.Tech Project Report**

# 1. Abstract

PDF documents frequently contain photographs, scanned figures, charts, graphs, diagrams and other visual information. Manually extracting these elements is time-consuming. This project proposes a Python-based PDF image extraction system that processes PDF pages, identifies embedded images, extracts them, and saves them as separate image files. PDF pages can also be rendered at high resolution so charts and vector graphics can be processed using image-processing techniques. OCR can be added for scanned documents.

# 2. Introduction

The project automates the collection of visual content from PDF documents and organizes the extracted files for further use. It is designed around PyMuPDF for PDF processing, Pillow for image handling, OpenCV for image analysis, and optional Tesseract OCR for scanned documents.

# 3. Problem Statement

Manual screenshot and copy-paste methods are slow, repetitive and may reduce image quality or include unwanted page content. An automated method is therefore required to extract embedded images and provide a process for charts, graphs and scanned pages.

# 4. Objectives

- Upload and process PDF documents.
- Read PDF pages programmatically.
- Extract embedded images.
- Save images as PNG/JPG files.
- Process charts and graphs rendered from PDF pages.
- Support scanned PDFs through rendering and optional OCR.
- Organize extracted content for viewing and downloading.

## 5. System Workflow

### PDF Image / Chart / Graph Extraction Workflow

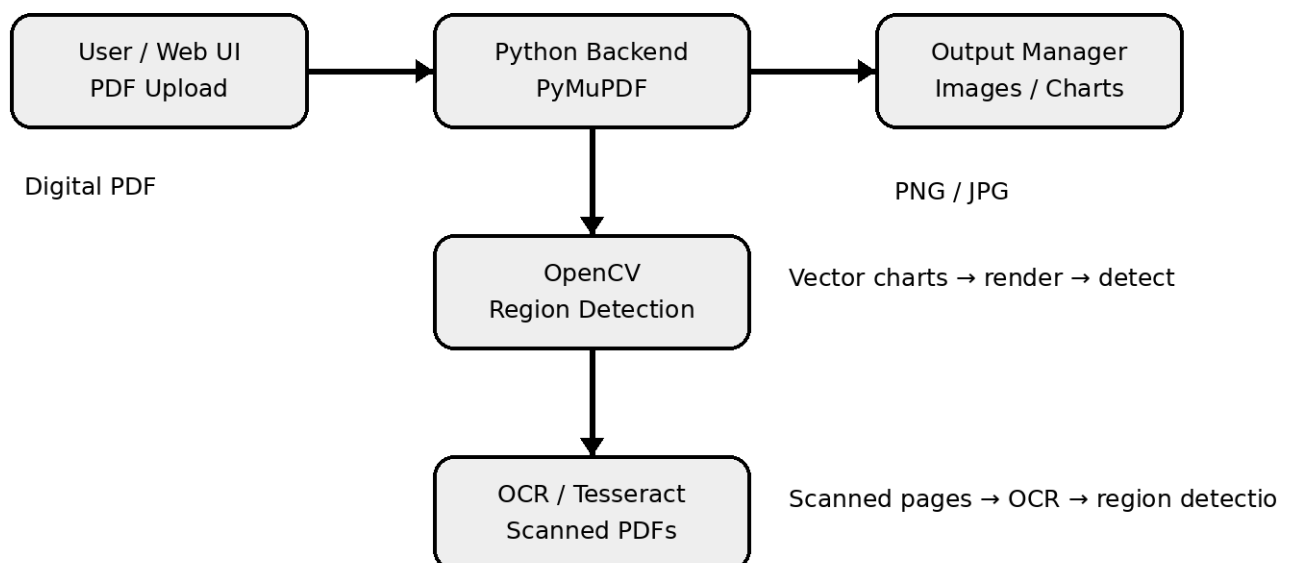
Embedded images are extracted directly; vector/scanned content can be rendered and processed.



The workflow follows the project specification: upload the PDF, analyze pages, find embedded images, extract them, render pages when chart/vector detection is needed, apply OpenCV/OCR where required, and finally display or download the results.

## 6. System Architecture

### System Architecture



The architecture separates PDF reading, computer-vision processing, OCR and output management. Embedded images can be extracted directly, while vector and scanned content can be rendered and

processed.

## 7. Technologies Used

| Technology          | Purpose                                |
|---------------------|----------------------------------------|
| Python              | Main programming language              |
| PyMuPDF (fitz)      | Read PDFs and extract embedded images  |
| Pillow              | Image handling and conversion          |
| OpenCV              | Image processing and region detection  |
| Tesseract OCR       | Text recognition for scanned documents |
| Flask / FastAPI     | Optional web backend                   |
| HTML/CSS/JavaScript | Optional frontend                      |

## 8. Embedded Image Extraction Module

```
import fitz
import os

pdf_path = "sample.pdf"
output_dir = "output/images"
os.makedirs(output_dir, exist_ok=True)

doc = fitz.open(pdf_path)

for page_number, page in enumerate(doc, start=1):
    images = page.get_images(full=True)

    for image_number, image in enumerate(images, start=1):
        xref = image[0]
        image_info = doc.extract_image(xref)
        ext = image_info["ext"]
        image_bytes = image_info["image"]

        filename = f"page_{page_number}_image_{image_number}.{ext}"
        with open(os.path.join(output_dir, filename), "wb") as f:
            f.write(image_bytes)

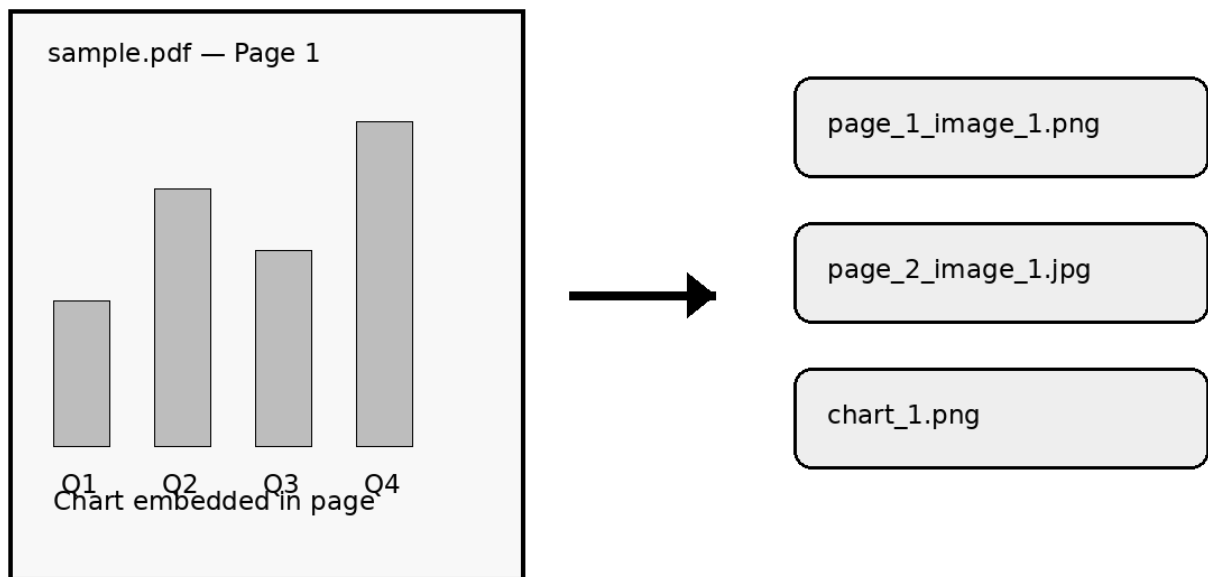
doc.close()
print("Image extraction completed.")
```

## 9. Chart and Graph Extraction

Charts and graphs may be stored as embedded images or generated from vector drawing commands. Embedded charts can be extracted directly. Vector charts can be rendered at high resolution and analyzed with OpenCV using line detection, contour detection, connected components and OCR/text analysis. The detected region can then be cropped and saved as an image.

## 10. Example Output

## Illustrative Extraction Result



The illustration shows the intended result: a PDF page is processed and visual elements are saved as separate files such as `page_1_image_1.png` and `chart_1.png`.

## 11. Scanned PDF Processing

A scanned PDF may contain page images rather than separately embedded figures. The page can be rendered or read as an image, OpenCV can identify regions of interest, and OCR can detect text and headings that help locate charts or figures.

## 12. Suggested Project Structure

```
Image-Extraction/  
  input/  
    sample.pdf  
  output/  
    images/  
    charts/  
    graphs/  
  app.py  
  extractor.py  
  detector.py  
  requirements.txt
```

## 13. Advantages

- Reduces manual extraction effort.
- Can process multiple pages automatically.
- Preserves embedded image data without taking screenshots.
- Can be extended for charts, graphs and scanned documents.
- Creates an organized output directory.
- Provides a foundation for AI-assisted document analysis.

## 14. Limitations

- Not every chart is stored as a separate image.
- Vector graphics may require page rendering and computer-vision processing.
- Scanned documents may require OCR and image segmentation.
- Complex layouts can produce false detections.
- Chart classification requires additional logic or machine learning.

## 15. Future Scope

Future versions can classify extracted content as photograph, chart, graph, table, diagram or logo. Deep-learning models can be added for visual-region detection. The system can also export detected charts as structured data, provide a browser dashboard, process PDFs in batches, and support cloud storage.

## 16. Expected Output

After processing a PDF, the application produces separate image files such as `page_1_image_1.png` and `page_2_image_1.jpg`. Detected charts and graphs can be stored in dedicated `charts/` and `graphs/` folders, with thumbnails and download options in a web interface.

## 17. Conclusion

The Image Extraction from PDF project provides an automated method for collecting visual content from PDF documents. PyMuPDF supports direct embedded-image extraction, while OpenCV and OCR extend the system to vector graphics and scanned documents. The project is suitable for academic and document-processing applications and can be expanded into an AI-based document understanding system.

## 18. References

- PyMuPDF documentation – PDF processing and image extraction.
- OpenCV documentation – computer vision and image processing.
- Pillow documentation – Python Imaging Library.
- Tesseract OCR documentation – optical character recognition.
- Python documentation – programming language and standard libraries.